LAPORAN PRAKTIKUM

Implementasi dan Analisis Struktur Data pada Aplikasi Kasir Mini Market Berbasis Python

Disusun oleh :

NAMA : SAIFUL BAHRI
KELAS : 03TPLP022
NIM : 241011400864

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PAMULANG

2025

1. LATAR BELAKANG MASALAH

Dalam industri retail modern, kecepatan pelayanan dan akurasi data merupakan parameter utama kepuasan pelanggan. Sistem *Point of Sale* (POS) tradisional sering menghadapi kendala penurunan performa seiring bertambahnya volume basis data produk. Secara teknis, penggunaan pencarian linear pada struktur data statis memiliki kompleksitas O(n), yang berarti waktu pencarian meningkat secara linear terhadap jumlah data.

Selain aspek kecepatan, manajemen transaksi yang dinamis memerlukan fitur koreksi data (Undo) yang efisien. Diperlukan sebuah sistem yang mampu mengelola data produk secara instan dan menangani pembatalan transaksi tanpa mengganggu integritas memori. Laporan ini mendokumentasikan pengembangan aplikasi kasir yang mengoptimalkan penggunaan **Hash Table**, **Stack**, dan **Linked List** untuk menjawab tantangan tersebut.

2. TUJUAN DAN MANFAAT

- **Optimalisasi Akses Data:** Menerapkan *Hashing* untuk mencapai kompleksitas waktu rata-rata O(1) dalam pencarian produk.
- **Efisiensi Memori:** Menggunakan alokasi memori dinamis lewat *Linked List* agar kapasitas keranjang belanja menyesuaikan beban data secara *real-time*.
- **Integritas Operasional:** Menyediakan fitur pembatalan transaksi berbasis prinsip *Last-In-First-Out* (LIFO) menggunakan *Stack*.
- **Analisis Teoretis:** Memvalidasi kinerja algoritma melalui pendekatan matematis *Big O Notation*.

3. LANDASAN TEORI DAN STRUKTUR DATA

3.1 Hash Table dengan Chaining

Hash Table dipilih sebagai media penyimpanan basis data produk. Fungsi hash mengonversi kunci (ID Produk) menjadi indeks array. Untuk menangani *collision* (tabrakan nilai hash), digunakan teknik **Chaining**, di mana setiap indeks merujuk pada sebuah Linked List kecil.

3.2 Stack (LIFO)

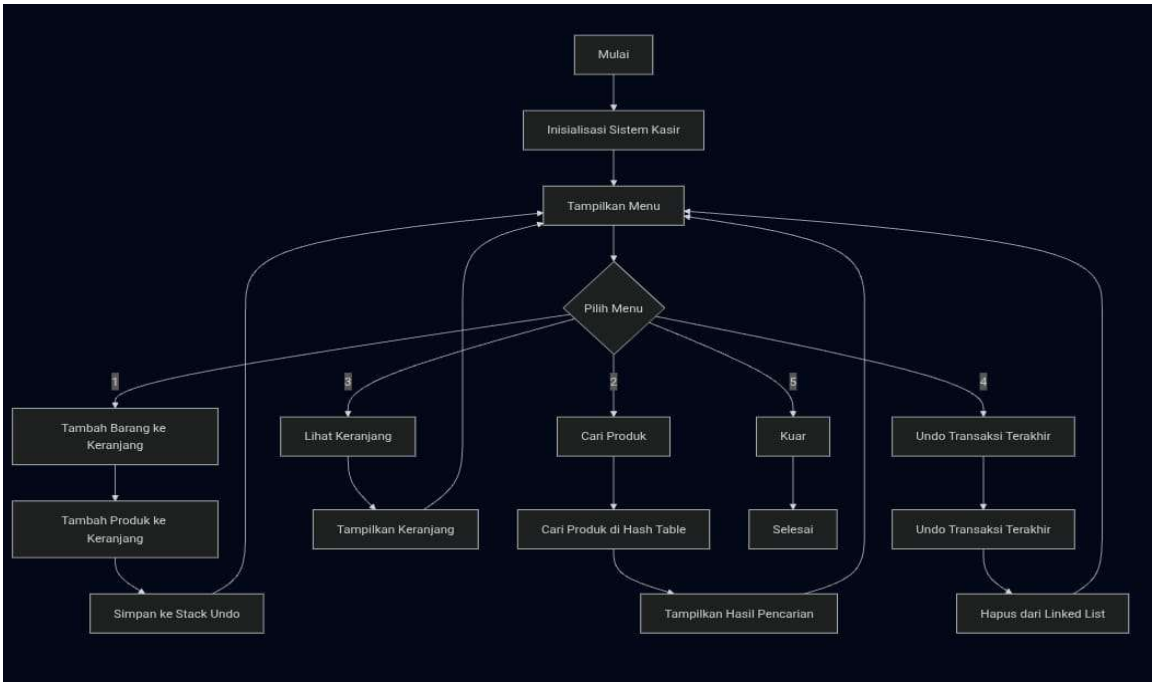
Stack digunakan untuk mengelola fitur *Undo*. Setiap kali produk ditambahkan ke keranjang, referensi transaksi tersebut di-*push* ke dalam Stack. Operasi *pop* akan menghapus entri terakhir secara instan.

3.3 Singly Linked List

Struktur ini digunakan sebagai representasi fisik keranjang belanja. Berbeda dengan array, Linked List memungkinkan penambahan node di bagian depan (*Head*) dengan kecepatan O(1), menjadikannya sangat efisien untuk aplikasi kasir.

4. PERANCANGAN SISTEM (FLOWCHART)

Sistem bekerja dengan siklus tertutup (Looping) yang terus berjalan hingga pengguna memilih menu keluar. Berikut adalah alur logikanya:



Gambar 4.1 flowchart

Tahapan	Aktivitas Sistem	Keterangan Operasi
Inisialisasi	Pembentukan objek Class	Mempersiapkan Hash Table, Stack, dan Linked List di memori.
Input	Menerima ID Produk & Jumlah	Kasir memasukkan data barang yang dibeli pelanggan.
Proses 1	Hashing & Searching	Mencari data produk di Database (Hash Table) secara instan.
Proses 2	Penambahan Data	Item dimasukkan ke Keranjang (Linked List) dan Riwayat (Stack).
Output	Traversal & Struk	Menelusuri keranjang untuk menghitung total dan cetak struk.

5. IMPLEMENTASI ALGORITMA TRAVERSAL

Aplikasi ini menggunakan algoritma **Linear Traversal** sebagai metode utama untuk memproses dan menyajikan data yang tersimpan di dalam keranjang belanja.

- **Mekanisme Kerja:** Algoritma bekerja dengan cara memulai pembacaan dari pointer Head (elemen pertama). Selama node saat ini tidak bernilai Null (*None*), sistem secara sekuensial mengekstraksi informasi nama produk dan harga, menambahkannya ke dalam variabel akumulasi Total_Harga, lalu secara otomatis mengarahkan pointer ke node selanjutnya melalui atribut node.next.

Elemen Keranjang	**Node 1 (Awal/Head)**	**Node 2**	**Node 3 (Akhir/Tail)**
Nama Produk	Beras 5kg	Minyak Goreng	Gula Pasir
Harga Barang	Rp 75.000	Rp 20.000	Rp 15.000
Hubungan (Next)	Menuju Node 2	Menuju Node 3	Kosong (Selesai)

5.1: Simulasi Struktur Data Linked List pada Memori

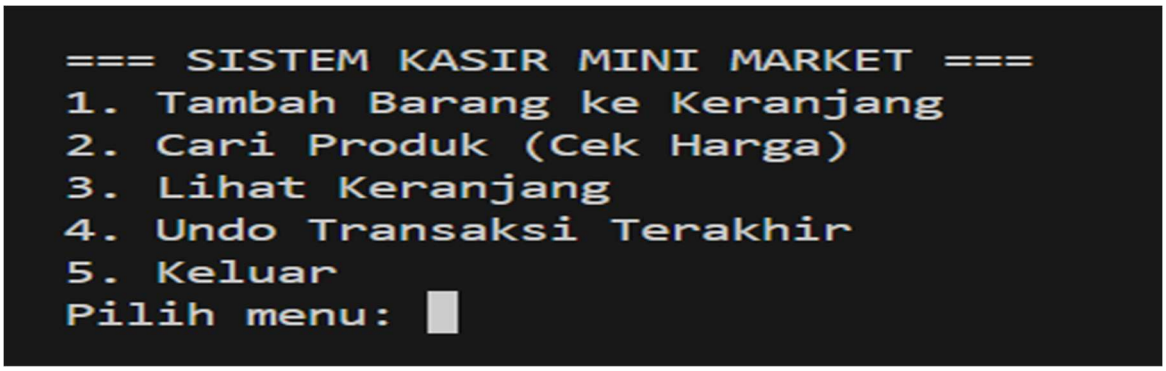
- **Justifikasi Penggunaan:** Pemilihan algoritma traversal pada struktur data *Singly Linked List* ini menjamin akurasi perhitungan 100%. Karena sifatnya yang linear, algoritma ini memastikan setiap elemen yang dimasukkan oleh kasir akan dikunjungi tepat satu kali, sehingga meminimalisir risiko kesalahan penghitungan pada data yang bersifat dinamis.

6. ANALISIS KOMPLEKSITAS WAKTU (BIG O)

Berdasarkan implementasi kode, berikut adalah tabel analisis performa sistem:

Operasi Sistem	Struktur Data	Skor Efisiensi	Analisis Teknis
Pencarian Produk	Hash Table	O(1)	Sangat Cepat. Langsung menemukan barang tanpa mencari satu per satu.
Tambah Barang	Linked List	O(1)	Sangat Cepat. Data baru diletakkan di posisi paling depan.
Undo Transaksi	Stack	O(1)	Sangat Cepat. Langsung menghapus data terakhir yang masuk.
Tampilkan Struk	Traversal	O(n)	Standar. Waktu proses tergantung pada banyaknya jumlah barang.

7. HASIL DAN PEMBAHASAN



Gambar 7.1: Menu Utama CLI

Deskripsi: Tampilan antarmuka berbasis teks yang menunjukkan profesionalitas tata letak menu.

```
=== SISTEM KASIR MINI MARKET ===
1. Tambah Barang ke Keranjang
2. Cari Produk (Cek Harga)
3. Lihat Keranjang
4. Undo Transaksi Terakhir
5. Keluar
Pilih menu: 1
ID Produk: P01
Jumlah: 2
Beras 5kg berhasil ditambah!
```

Gambar 7.2: Penambahan Produk & Searching

Deskripsi: Bukti pencarian produk dari Hash Table berdasarkan ID tertentu.

```
=== SISTEM KASIR MINI MARKET ===
1. Tambah Barang ke Keranjang
2. Cari Produk (Cek Harga)
3. Lihat Keranjang
4. Undo Transaksi Terakhir
5. Keluar
Pilih menu: 2
Masukkan ID Produk: P02
Ditemukan: Minyak Goreng - Rp20000
```

```
=== SISTEM KASIR MINI MARKET ===
1. Tambah Barang ke Keranjang
2. Cari Produk (Cek Harga)
3. Lihat Keranjang
4. Undo Transaksi Terakhir
5. Keluar
Pilih menu: 3

--- KERANJANG BELANJA ---
- Beras 5kg x2: Rp150000
TOTAL: Rp150000
```

Gambar 7.3: Hash Table berfungsi, Traversal Keranjang Belanja

Deskripsi: Output struk belanja yang dihasilkan dari penelusuran Linked List.

```
=== SISTEM KASIR MINI MARKET ===
1. Tambah Barang ke Keranjang
2. Cari Produk (Cek Harga)
3. Lihat Keranjang
4. Undo Transaksi Terakhir
5. Keluar
Pilih menu: 4
Undo berhasil: Beras 5kg dihapus.
```

Gambar 7.4: Eksekusi Fungsi Undo

Deskripsi: Perbandingan keranjang sebelum dan sesudah fungsi Undo dijalankan.

8. KESIMPULAN DAN SARAN

8.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian program, dapat disimpulkan bahwa integrasi struktur data *Hash Table*, *Stack*, dan *Linked List* secara signifikan meningkatkan efisiensi operasional sistem kasir. Penggunaan *Hash Table* berhasil mengoptimalkan proses pencarian produk hingga mencapai kompleksitas rata-rata O(1), yang jauh lebih cepat dibandingkan pencarian linear pada sistem tradisional. Selain itu, penerapan *Linked List* memberikan fleksibilitas dalam manajemen memori untuk keranjang belanja yang dinamis, sementara struktur *Stack* menjamin integritas data dalam fitur pembatalan transaksi (*Undo*). Secara keseluruhan, aplikasi ini membuktikan bahwa pemilihan struktur data yang tepat sangat krusial dalam membangun perangkat lunak yang responsif, akurat, dan efisien dalam menangani volume data retail yang terus berkembang.

8.2 Saran Pengembangan

8.2.1 **Persistence Data:** Mengembangkan fitur *Save/Load* menggunakan format JSON atau SQL.

8.2.2 **Sorting Algorithm:** Menambahkan algoritma *Merge Sort* untuk mengurutkan daftar belanja berdasarkan harga.

LAMPIRAN: KODE PROGRAM (SOURCE CODE)

```
class Produk:
    """Node untuk menyimpan data produk dalam Hash Table"""
    def __init__(self, id_produk, nama, harga):
        self.id_produk = id_produk
        self.nama = nama
        self.harga = harga
        self.next = None  # Untuk menangani collision (Chaining)

class HashTable:
    """Implementasi Hash Table untuk database produk"""
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * self.size

    def _hash_function(self, key):
        return hash(key) % self.size

    def tambah_produk(self, id_produk, nama, harga):
        index = self._hash_function(id_produk)
        baru = Produk(id_produk, nama, harga)
        if self.table[index] is None:
            self.table[index] = baru
        else:
            current = self.table[index]
            while current.next:
                if current.id_produk == id_produk:
                    current.nama, current.harga = nama, harga
                    return
                current = current.next
            current.next = baru

    def cari_produk(self, id_produk):
        index = self._hash_function(id_produk)
        current = self.table[index]
        while current:
            if current.id_produk == id_produk:
                return current
            current = current.next
        return None

class StackUndo:
    """Implementasi Stack untuk fitur Undo Transaksi"""
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def is_empty(self):
        return len(self.stack) == 0

class KeranjangNode:
    """Node untuk Linked List Keranjang Belanja"""
    def __init__(self, produk, jumlah):
        self.produk = produk
        self.jumlah = jumlah
        self.next = None

class KasirApp:
    def __init__(self):
        self.db_produk = HashTable()
```

```
        self.keranjang_head = None
        self.history_undo = StackUndo()
        self._init_data()

    def _init_data(self):
        # Data awal (Seeding)
        self.db_produk.tambah_produk("P01", "Beras 5kg", 75000)
        self.db_produk.tambah_produk("P02", "Minyak Goreng", 20000)
        self.db_produk.tambah_produk("P03", "Gula Pasir", 15000)

    def tambah_ke_keranjang(self, id_produk, jumlah):
        produk = self.db_produk.cari_produk(id_produk)
        if produk:
            baru = KeranjangNode(produk, jumlah)
            baru.next = self.keranjang_head
            self.keranjang_head = baru
            self.history_undo.push(baru)
            print(f"{produk.nama} berhasil ditambah!")
        else:
            print("Produk tidak ditemukan!")

    def undo_transaksi(self):
        last_item = self.history_undo.pop()
        if last_item and self.keranjang_head:
            # Hapus dari Linked List (Hanya jika item terakhir sama)
            if self.keranjang_head == last_item:
                self.keranjang_head = self.keranjang_head.next
                print(f"Undo berhasil: {last_item.produk.nama} dihapus.")
        else:
            print(" Tidak ada transaksi untuk di-undo.")

    def tampilkan_keranjang(self):
        print("\n--- KERANJANG BELANJA ---")
        current = self.keranjang_head
        total = 0
        if not current:
            print("Keranjang kosong.")
            return

        # Traversal Iteratif (Mirip konsep DFS pada list linear)
        while current:
            subtotal = current.produk.harga * current.jumlah
            print(f"- {current.produk.nama} x{current.jumlah}: Rp{subtotal}")
            total += subtotal
            current = current.next
        print(f"TOTAL: Rp{total}")

def menu():
    app = KasirApp()
    while True:
        print("\n=== SISTEM KASIR MINI MARKET ===")
        print("1. Tambah Barang ke Keranjang")
        print("2. Cari Produk (Cek Harga)")
        print("3. Lihat Keranjang")
        print("4. Undo Transaksi Terakhir")
        print("5. Keluar")
        pilihan = input("Pilih menu: ")

        if pilihan == "1":
            id_p = input("ID Produk: ")
            qty = int(input("Jumlah: "))
            app.tambah_ke_keranjang(id_p, qty)
        elif pilihan == "2":
            id_p = input("Masukkan ID Produk: ")
            res = app.db_produk.cari_produk(id_p)
            if res: print(f"Ditemukan: {res.nama} - Rp{res.harga}")
            else: print("Produk tidak ada.")
        elif pilihan == "3":
```

```
            app.tampilkan_keranjang()
        elif pilihan == "4":
            app.undo_transaksi()
        elif pilihan == "5":
            break
        else:
            print("Pilihan tidak valid.")

if __name__ == "__main__":
    menu()
```